

Lecture Material for
CSE:4409 Database Management 2 (V 2.0)
Partial

This document has been prepared from various textbooks, official documentation of the database (i.e. Postgre), other internet sources and practical experience. References will be mentioned after the lecture note is finished.

Prepared by:
Dr. Abu Raihan Mostofa Kamal
Professor, Department of Computer Science and Engineering (CSE), IUT
Contact: raihan.kamal@iut-dhaka.edu

Department of CSE
Islamic University of Technology
Bangladesh
Updated On: June 15, 2026

Contents

1	Contents Revisited	1
1.1	Following are the tentative outline	1
2	Contents Revisited	2
2.1	Following are the tentative outline	2
3	Prerequisite: Database Installation	3
3.1	Installation	3
3.2	Connect to Database	3
3.2.1	Directly to Database	3
3.2.2	Connection to Database as Other user (than OS user-name)	5
3.2.3	Connection to Database via pgAdmin tool	6
4	PL/ pgSQL Basics	8
4.1	What is PL/pgSQL?	8
4.1.1	Benefits of PL/pgSQL	8
4.1.2	Installation	9
4.2	PL/pgSQL Block	9
4.2.1	Anonymous or unnamed blocks	9
4.2.2	Named Blocks	10
4.3	Basic Datatype	11
4.3.1	Number	11
4.3.2	Character Types	11
4.3.3	Arrays	12
4.3.4	Date/Time Types	13
4.3.5	Network Address	15
4.3.6	Range Type	18
4.3.7	GiST Indexes for Range Types	23
4.3.8	Composite Data Type	26
4.3.9	Domain Type	28
5	Control Structure	30
5.1	IF/ELSE Statement	30
5.2	CASE Statement	31

5.3	LOOP Statement	33
5.4	FOR Statement	34
5.4.1	Use Case for FOR loop	35
5.5	Control Statements in PL/pgSQL: Best Practices	35
6	Functions and Procedures	36
6.1	Benefits of Functions and Procedure	36
6.2	Functions	37
6.2.1	%TYPE, %ROWTYPE and RECORD Data-types	40
6.2.2	Return More Rows	43
7	Exception Handling in PL/pgSQL	45
7.1	Built-in Exceptions	45
7.2	User-defined Exceptions	48
8	Cursor to Handle Selected Records	50
8.1	Implicit Cursor	50
8.2	Explicit Cursor	52
8.2.1	Cursor FOR Loop	55

Chapter 1

Contents Revisited

1.1 Following are the tentative outline

- Database basic constructs: Table, Key, index, view, mapping cardinality
- PL/pgSQL basics:
 - Benefits of Procedural SQL such as PL/pgSQL
 - Blocks
 - Variables
 - Basic data types
 - Arrays
 - Control statements and iteration
 - More on Arrays, JSON datatype
 - Database Cursors
 - Datatype Casting
 - Dynamic SQL
 - Built-in Functions and Procedures
- Functions and Procedures
- Exception Handling
- Triggers

Chapter 2

Contents Revisited

2.1 Following are the tentative outline

- Database basic constructs: Table, Key, index, view, mapping cardinality
- PL/pgSQL basics:
 - Benefits of Procedural SQL such as PL/pgSQL
 - Blocks
 - Variables
 - Basic data types
 - Arrays
 - Control statements and iteration
 - More on Arrays, JSON datatype
 - Database Cursors
 - Datatype Casting
 - Dynamic SQL
 - Built-in Functions and Procedures
- Functions and Procedures
- Exception Handling
- Triggers

Chapter 3

Prerequisite: Database Installation

Installation of Postgre Database is very easy especially in Linux-based system.

3.1 Installation

- For database installation on Redhat Linux Family you can visit:
<https://www.postgresql.org/download/linux/redhat/>
- For Ubuntu based system:
<https://www.postgresql.org/download/linux/ubuntu/>

3.2 Connect to Database

Once the database is installed you can connect to it as postgres user (already exists in the system):
Issue the following command from the bash terminal

3.2.1 Directly to Database

Step 1: Connect as an existing user:

1. Connect as postgres user by: `sudo -u postgres psql`
2. Next you should set a password for postgres user by `\password postgres`
Type your new password (used **test123**), press Enter, and retype it to confirm. Your keystrokes will be completely hidden from the screen for security.
3. Enable password authentication method to `scram-sha-256` (If needed) see 3.2.2 for details (how to login as user other than OS user)
4. After setting password authentication: database will ask you 2 passwords

✓ **OS User:** `[sudo] password for raihan:`

This is your OS password for the user raihan

✓ **Database User:** `Password for user postgres:`

It is the password just you have set in previous user by `\password postgres`

It will take you to the `postgres=#` SQL prompt or simply database terminal

Additional Actions needed from postgres user from postgres user (database admin):

(these actions must be performed as postgres user)

- Change the password of another user:

```
alter user raihan with password 'raihan123';
```

- Get the list of all databases a specific user owns from postgres user (database admin):

```
1 SELECT datname
2 FROM pg_database
3 JOIN pg_user ON pg_database.datdba = pg_user.usesysid
4 WHERE pg_user.username = 'raihan';
```

Alternatively: You can do it from any user to get all users and database summary by:

```
\l
```

Step 2: Create a new user

Here you have to create both user and a workable database (in postgres database is like one single application, one user may be owner of a number of databases/applications). This is more natural design compared to Oracle concept.

```
1 CREATE USER raihan WITH PASSWORD 'raihan123';
2
3 CREATE DATABASE dbiut OWNER raihan;
```

Step 3: Connect as the new user (from bash terminal)

```
psql -U raihan -d dbiut
```

(Note: copy past may not work, better not to use any underscore symbol in any naming rather type them)

3.2.2 Connection to Database as Other user (than OS user-name)

```
1  --log in as postgres
2  sudo -u postgres psql;
3
4  --- now create another user: not identical to OS user
5  create user armk with password 'armk123';
6  create database dbarmk owner armk;
7
8  ---disconnect
9  \q;
10 ---connect as this new user
11 psql -U armk -d dbarmk;
```

This will create connection error like:

```
psql: error: connection to server on socket
"/run/postgresql/.s.PGSQL.5432" failed: FATAL:
Peer authentication failed for user "armk"
```

Solution:

Option A: (temporary) Just bypass the local Unix Socket by adding -h flag:

```
psql -h localhost -U armk -d dbarmk;
```

Option B: (permanent solution)

Modify the configuration file. Before that you can locate the file by the following command from database terminal:

```
SHOW hba_file;
```

Now edit the file by:

```
sudo nano /var/lib/pgsql/18/data/pg_hba.conf
```

Change the local line from peer to scram-sha-256

```
1 # TYPE DATABASE USER  ADDRESS METHOD
2
3 #modified version (peer has been repaced with scram-sha-256)
4 # "local" is for Unix domain socket connections only
5 local all          all          scram-sha-256
```

Now need to restart the Postgre Database.

1. Find the name of the service:

```
systemctl list-units | grep postgres
```

2. Possible Output:

```
1 session-c8.scope loaded active running Session c8 of User postgres
2 postgresql-18.service loaded active running PostgreSQL 18 database server
```

So, the name of the db server (process) in OS is postgresql-18.service

Now restart by:

```
sudo systemctl restart postgresql-18.service;
```

3. You can log in by:

```
psql -U armk -d dbarmk
```

3.2.3 Connection to Database via pgAdmin tool

pgAdmin is a Feature-rich, open-source administration and development platform for PostgreSQL.

Step 1: Installation: Follow the steps here

<https://www.pgadmin.org/download/pgadmin-4-rpm/>

Postgre Database has a default user called postgres

Before that it is **better to set a password** for this user (which will be needed to connect from pgAdmin tool later)

Just do it from bash terminal:

```
sudo -i -u postgres psql -c "ALTER USER postgres WITH PASSWORD 'test123';"
```

Step 2: Enable Password Authentication Once you are connected to database as postgres user check the location of its configuration file by:

```
show hba_file;
```

It should show output as follows:

```
          hba_file
-----
/var/lib/pgsql/18/data/pg_hba.conf
(1 row)
```

Now edit (you can use nano or vi) the file by:

```
sudo gedit /var/lib/pgsql/18/data/pg_hba.conf
```

Now find similar to :

```
# IPv4 local connections: #all has been replaced by raihan
host    all             all             127.0.0.1/32      scram-sha-256
```

Add or modify as

```
# IPv4 local connections: #all has been replaced by raihan
host    all             all             127.0.0.1/32      scram-sha-256
host    all             raihan          127.0.0.1/32      scram-sha-256
```

Now you can open pgAdmin and create a server and configure it supplying username as (raihan) password as you have already altered, database name as you have created earlier and hostname as local host i.e. 127.0.0.1

Chapter 4

PL/ pgSQL Basics

4.1 What is PL/pgSQL?

PostgreSQL uses SQL (Structured Query Language) as a **default** query language. SQL is a common domain-specific language for relational databases.

PostgreSQL uses some **extensions and features** to implement the standards of SQL. In addition to SQL, PostgreSQL supports many procedural languages like PL/pgSQL, PL/Java, PLV8, PL/Python, PL/Perl, etc. Using these languages, you can create functions, stored procedures, and triggers which will improve the performance by reducing the multiple iterations to the databases.

PL/pgSQL is a procedural extension for PostgreSQL. It allows developers to combine SQL with **procedural logic** such as loops, conditions, variables, and control structures. This language is used to create:

- User-defined functions
- Stored procedures
- Triggers

4.1.1 Benefits of PL/pgSQL

- **Integration with SQL:** Execute SQL commands seamlessly within procedural blocks.
- **Flexibility:** Supports custom logic for complex operations.
- **Re-usability:** Functions and triggers written in PL/pgSQL can be reused across multiple systems/users
- **Easy Code Migration:** PL/pgSQL is portable across different operating systems and platforms. This makes it easier to migrate code between different environments.
- **Performance:**

- Improve data processing speed by using precompiled code through stored procedures which will be faster than raw SQL queries.
- Extra round trips between client and server are eliminated
- Intermediate results that the client does not need do not have to be marshaled or transferred between server and client
- **Server-side execution:** PL/pgSQL code is executed on the server side, which means that it has direct access to the database and can perform database operations more efficiently than client-side code.
- **Centralizing the business logic** at the database layer for consistency.

4.1.2 Installation

PL/pgSQL is already included in PostgreSQL. So if you have installed PostgreSQL on your machine, PL/pgSQL is already there.

Check the status: You can check the status by simply executing `\dx`. It will show List of installed extensions.

4.2 PL/pgSQL Block

PL/pgSQL is a block-structured language, therefore, a PL/pgSQL function or store procedure is organized into blocks.

The **basic unit** in any PL/pgSQL code is a block. All PL/pgSQL code is composed of a single block or blocks that occur either sequentially or nested within another block.

There are 2 types of Blocks:

1. Anonymous or unnamed blocks (DO)
2. Named blocks (functions)

4.2.1 Anonymous or unnamed blocks

Anonymous or unnamed blocks are generally constructed dynamically and executed only once by the user. It is generally used for testing of different sub-program at time of development.

General Structure of Anonymous Blocks

```
1 DO $$  
2   [ <<label>> ]  
3   [ DECLARE  
4     -- Variable declaration here  
5   ]  
6   BEGIN
```

```

7  -- Execute statements here
8  END [ label ];
9  $$;

```

Few Examples of Hello World Programs:

```

1  DO
2  $$
3  BEGIN
4  RAISE NOTICE 'Hello World From Postgre Database';
5  END;
6  $$;

```

```

1  DO
2  $$
3  DECLARE
4  X NUMERIC:=102.32;
5  Y NUMERIC:=23.43;
6  BEGIN
7  RAISE NOTICE 'Hello World: THE VALUE OF X IS % AND Y IS %',X,Y;
8  RAISE NOTICE 'Hello World: Their sum is %',X+Y;
9  END;
10 $$;

```

4.2.2 Named Blocks

Named blocks have a name associated with them, are stored in the database, can be executed repeatedly, and can take in parameters.

Example:

```

1  CREATE OR REPLACE FUNCTION factorial(num INTEGER)
2  RETURNS INTEGER AS $$
3  DECLARE
4  result INTEGER := 1;
5  BEGIN
6  FOR i IN 1..num LOOP
7  result := result * i;
8  END LOOP;
9  RETURN result;
10 END;
11 $$ LANGUAGE plpgsql;

```

Now call the function as

```
select factorial(7);
```

You can add label to any part of the code for better readability. For instance the same code can be written as:

```

1  CREATE OR REPLACE FUNCTION factorial(num INTEGER)

```

```
2 RETURNS INTEGER AS $$
3 DECLARE
4 result INTEGER := 1;
5 BEGIN
6 <<factorial_loop>>
7 FOR i IN 1..num LOOP
8 result := result * i;
9 END LOOP factorial_loop;
10 RETURN result;
11 END;
12 $$ LANGUAGE plpgsql;
```

4.3 Basic Datatype

For more details you are advised to find [Documentation PostgreSQL 18 chapter 8](#)

There are many types of data found in Postgre SQL. Major types are as follows:

1. Number
2. Character
3. Date-time
4. Boolean
5. Array
6. Network Address
(<https://blog.stackademic.com/postgresql-specific-network-address-data-type/>)
7. Range Type
8. Domain Type (it is a constraint)

4.3.1 Number

Numeric types consist of two-, four-, and eight-byte integers, four- and eight-byte floating-point numbers, and selectable-precision decimals.

Summary Table:

4.3.2 Character Types

`varchar(n)` or `text` can be used.

Although the `text` type is not in the SQL standard, several other SQL database management systems have it as well. `text` is PostgreSQL's native string data type.

Name	Storage	Description	Range
smallint	2 bytes	small-range integer	-32768 to +32767
integer	4 bytes	typical choice for integer	-2147483648 to +2147483647
bigint	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807
decimal/numeric	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
real	4 bytes	variable-precision, inexact	6 decimal digits precision
smallserial	2 bytes	small autoincrementing integer	1 to 32767
serial	4 bytes	autoincrementing integer	1 to 2147483647

4.3.3 Arrays

Arrays are a complex data type in PL/pgSQL, but they can be incredibly useful for working with lists of data. To declare an array variable, you can use the ARRAY data type. Here is the syntax:

```

1 DECLARE
2   my_array INTEGER[];
```

You can initialize an array with values using curly braces:

```

1 DECLARE
2   my_array INTEGER[] := '{1, 2, 3}';
```

You can access individual elements of an array using square brackets. The following is a simple example:

```

1 DO
2   $$
3   DECLARE
4   my_array INTEGER[] := '{1, 2, 3}';
5
6   second_element INTEGER;
7   BEGIN
8   second_element := my_array[2];
9   RAISE NOTICE 'second element from my array is: %', second_element;
10  END;
11  $$;
```

You can also loop over the elements of an array using a FOR loop. The following is an example:

```

1 DO
2   $$
3   DECLARE
```

```
4  my_array INTEGER[] := '{1, 2, 3}';
5  total INTEGER := 0;
6  BEGIN
7  FOR i IN 1..array_length(my_array, 1) LOOP
8  total := total + my_array[i];
9  END LOOP;
10 RAISE NOTICE 'total of the elements in my array is: %', total;
11 END;
12 $$;
```

Note: Here `array_length()` function requires **Two Arguments** because PostgreSQL supports multidimensional arrays.

Dimension 1: The number of rows (or elements in a standard list).

Dimension 2: The number of columns.

4.3.4 Date/Time Types

You can use `timestamp` or `date` or `time` with timezone options or without it.

4.3.4.1 PostgreSQL DATE Arithmetic

PostgreSQL's date arithmetic functions allow you to perform calculations with date and time values, such as adding or subtracting intervals from dates. These functions are essential for operations that require dynamic date manipulations, like calculating future or past dates.

Example:

1. Adding Days to a Date

```
1  -- adding 5 days
2  SELECT '2023-10-15'::date + interval '5 days';
3
4  SELECT CURRENT_DATE + interval '5 days';
5
6
7  SELECT '2023-10-15'::date + interval '2 months';
8
9  SELECT CURRENT_DATE + interval '2 months';
10
11
12 --- Adding with mixed up parts
13
14 SELECT '2023-10-15'::date + interval '1 year 3 months 10 days';
15
16 SELECT CURRENT_DATE + interval '1 year 8 months 14 days';
17
```

```

18 SELECT CURRENT_TIMESTAMP + interval '1 year 8 months 14 days';
19
20
21 SELECT TO_CHAR(CURRENT_TIMESTAMP + interval '1 year 8 months 14
    days', 'Month DD YYYY');

```

or more readability you can use TO_CHAR with different format masks, as follows

```
SELECT TO_CHAR(CURRENT_DATE + INTERVAL '7 days', 'Month DD YYYY');
```

2. Extracting Parts of a Date:

Use EXTRACT to get different part of a date.

```

1
2  -- for current time
3
4  SELECT EXTRACT(YEAR FROM CURRENT_DATE);
5  -- Or, if you prefer the function syntax:
6  SELECT EXTRACT(YEAR FROM NOW());
7
8  --- for a given time
9
10 SELECT EXTRACT(YEAR FROM TIMESTAMP '2026-04-20'); -- Result: 2026
11 SELECT EXTRACT(MONTH FROM TIMESTAMP '2026-04-20'); -- Result: 4

```

3. AGE function:

There are two primary ways to use the function:

- (a) AGE(timestamp): Subtracts the given timestamp from the current date (at mid-night).
- (b) AGE(timestamp1, timestamp2): Subtracts the second timestamp from the first.

```

1
2  --- easy syntax for given date to now
3  SELECT AGE('1977-11-01'::date);
4
5
6  -- Calculates age based on current date
7  SELECT AGE(timestamp '2000-01-01');
8
9  SELECT AGE('2024-01-01', '1990-05-15');
10
11 SELECT AGE('2026-01-01', '2023-10-21');
12
13 SELECT AGE(TIMESTAMP '2026-01-01', TIMESTAMP '2023-10-21');
14
15 ---Use in a WHERE clause
16

```

```

17 SELECT name FROM users
18 WHERE AGE(birth_date) >= interval '18 years';

```

Notes on Type Casting:

The SQL Standard Literal :

```
SELECT TIMESTAMP '2026-01-01';
```

is similar to

```
SELECT '2026-01-01'::TIMESTAMP;
```

☑ Date in SQL where clause

```

1  -- Records from the last 7 days
2  SELECT * FROM users
3  WHERE created_at >= NOW() - INTERVAL '7 days';
4
5
6  -- Records from the last 1 hour
7  SELECT * FROM error_logs
8  WHERE log_time >= NOW() - INTERVAL '1 hour';

```

4.3.5 Network Address

Background on Network Address:

1. INET (The "Where")

When you see inet (often in the output of the ip addr command in Fedora), it represents the Interface Address. It is the specific identity of your machine on the network.

Format: A standard IP address (e.g., 192.168.1.15).

Purpose: It tells the network exactly where to send data packets destined for your computer.

2. CIDR (The "How Big")

CIDR is a notation used to describe a block of IP addresses and their associated routing prefix. It replaced the old "Class A, B, and C" system to be more efficient with IP space.

Format: An IP followed by a slash and a number (e.g., /24).

The Number: This represents how many bits are "masked" (fixed) for the network portion. A /24 means the first 24 bits are the network ID, leaving 8 bits for host addresses ($2^8 = 256$ addresses). A /32 means a single, specific IP address.

Note: ----/x here **max** value for x is **32**

PostgreSQL network data types:

- cidr for IPv4 and IPv6 networks.

- `inet` for IPv4 and IPv6 hosts and networks.
- `macaddr` for MAC addresses.

Example:

```

1  create table networking_sample (
2      ip_cidr cidr,
3      ip_inet inet,
4      mac_macaddr macaddr
5  );
6
7  insert into networking_sample VALUES
8      ('10.20.30.101', '10.20.30.101/24', '00-0c-29-8f-ca-0a');
9
10 INSERT INTO networking_sample (ip_column, cidr_column, mac_column) VALUES
11 -- Record 1: Standard IPv4 Host and Subnet
12 ('192.168.1.50', '192.168.1.0/24', '08:00:27:01:23:45'),
13
14 -- Record 2: IPv6 Global Unicast Address
15 ('2001:db8:85a3::8a2e:370:7334', '2001:db8:85a3::/64', '00:15:5d:01:ca:b2'),
16
17 -- Record 3: IPv4 Loopback/Local
18 ('127.0.0.1', '127.0.0.0/8', '00:00:00:00:00:00'),
19
20 -- Record 4: IPv6 Link-Local Address
21 ('fe80::20c:29ff:fe8f:ca0a', 'fe80::/64', '00:0c:29:8f:ca:0a'),
22
23 -- Record 5: IPv4 Public IP with a specific /32 mask
24 ('8.8.8.8', '8.8.8.0/24', '44:33:44:33:44:33');
25
26 ---- end of insert----
27
28
29 -- you can not insert gargabe values / invalid values like:
30 insert into networking_sample VALUES
31 ('10.20.30.101', '10.20.30.101/84', '00-0c-29-8f-ca-0a');
32
33 -- you can not insert text data (used o instead of 0):
34 insert into networking_sample VALUES
35 ('10.20.30.1o1', '10.20.30.101/24', '00-0c-29-8f-ca-0a');
36
37 ---but it works
38 insert into networking_sample VALUES
39 ('10.20.34.161', '10.20.30.101/32', '00-0c-29-8f-ca-0a');
40
41 --- will not work: as max value for /x is 32
42 insert into networking_sample VALUES
43 ('10.20.34.161', '10.20.30.101/33', '00-0c-29-8f-ca-0a');
44
45
46 -- it can classify correctly--
47 select family(ip_cidr)

```

```

48  from networking_sample;
49
50  --- now if you select---
51  select * from networking_sample;
52
53  Notice: Subnet /32 was added to the cidr data type by default, meaning that it
    is a single host

```

✔ Why do we need Network Address instead of text?

Here is the breakdown of why you should avoid storing IP addresses as text:

- Built-in Validation (No "Garbage" Data):

If you use a TEXT field, you can accidentally store invalid data like "192.168.1.300" or "apple". PostgreSQL's network types strictly enforce format. If you try to insert an invalid IP, the database will reject it immediately.

INET: Handles single hosts and networks (IPv4 and IPv6).

CIDR: Specifically for network masks (ensures that non-zero bits do not appear to the right of the mask).

It can classify correctly ether IPv4 or IPv6:

```

1  select ip_inet,family(ip_cidr)
2  from networking_sample;
3
4  --OUTPUT
5
6  ip_inet      | family
7  -----+-----
8  10.20.30.101/24 |      4
9  192.168.1.0/24  |      4
10 2001:db8:85a3::/64 |      6
11 127.0.0.0/8      |      4
12 fe80::/64        |      6
13 8.8.8.0/24       |      4
14 (6 rows)

```

- Specialized Search Operators:

This is the most significant technical advantage. With TEXT, you can only do exact matches or prefix searches (like LIKE '192.168.1.%'). With INET, you get powerful subnet logic:

Example: << implies Is Contained By Example: ip << '192.168.1.0/24'

```

1  SELECT * FROM access_logs
2  WHERE client_ip << '172.16.0.0/12';

```

Doing this with string manipulation or BETWEEN logic is slow, error-prone, and doesn't account for IPv6.

- Sorting and Storage Efficiency:
 - **Binary Storage:** PostgreSQL stores INET and CIDR data as binary values, not as strings. This makes them significantly smaller on disk, especially for IPv6.
 - **Logical Sorting:** If you sort TEXT, "10.0.0.2" comes after "10.0.0.10" because strings are sorted character-by-character. Network types sort numerically, so "10.0.0.2" correctly comes first.

4.3.6 Range Type

Installation Step: `CREATE EXTENSION IF NOT EXISTS btree_gist;` may not work.

Solution: `sudo dnf install postgresql18-contrib`

Purpose of Range Type:

Range types represent a span of values (e.g., dates, timestamps, or numbers) in a single database column. They are useful for scheduling, session tracking, and scientific measurements because they allow for efficient "overlap" and "containment/member-ship" queries.

Syntax: A range is defined using brackets `[]` for inclusive boundaries and parentheses `()` for exclusive boundaries.

Example:

``[a, b]` : a <= x <= b`

``[a, b)` : a <= x < b (Standard for many types)`

``(a, b)` : a < x < b`

Built-in Range Types: PostgreSQL provides several built-in range types ready to use:

Example: Postgre maintains **canonical** form for range datatypes:

That canonical form is:

👉 **lower bound inclusive, upper bound exclusive** i.e. `[ )`

`-- we write--`

`SELECT '[1, 10] '::int4range;`

`Ans: [1,11)`

`SELECT '(1, 10)'::int4range;`

`Ans: [2,10)`

Type	Description	Example
int4range	Range of integer	[1, 100)
int8range	Range of big integer	[1, 10000000000)
numrange	Range of numeric	[1.5, 9.9]
tsrange	Range of timestamp without time zone (Operates at the microsecond level)	[2026-05-04 10:00, 2026-05-04 11:00]
daterange	Range of date (Operates at the day level)	[2026-05-04, 2026-05-10]

✔ Why does this happen?

PostgreSQL canonicalizes ranges for discrete types like integers (int4range)

PostgreSQL stores ranges in a standard canonical form

✔ What [1,10] means mathematically?

[1, 10] means a range of {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

PostgreSQL rewrites it as: [1, 11)

Because:

[1, 11) = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

Recall: lower bound inclusive, upper bound exclusive

But for numbers!

```
SELECT '[1.0, 10.0]':::numrange;
```

```
--response is--
```

```
[1.0,10.0]
```

✔ Why?

Between 10 and anything larger, there are infinitely many values

(such as 10.0001, 10.000021 ...)

Discrete vs. Continuous

For a continuous range (like numrange), [1, 10] is different from [1, 11) because there are infinitely many numbers between 10 and 11 (like 10.5).

However, for discrete types like integers (int4range):

There is no integer between 10 and 11.

Therefore, "all integers from 1 to 10 inclusive" is mathematically identical to "all integers from 1 starting at 1 and ending before 11."

Range Operators: There are Comparison and Bound Extraction Functions.

✔ **Comparison:**

```
1 SELECT '[1, 10)']::int4range = '[1, 10)']::int4range; -- true
2 SELECT '[1, 10)']::int4range = '(1, 10)']::int4range; -- false
3
4 SELECT '[1, 5)']::int4range < '[2, 8)']::int4range; --true
5 SELECT '[1, 5)']::int4range < '(2, 5)']::int4range; --true
```

Consider a series: S1: {1,2,3,4,5,6,7,8} Now another series S2={3,4,5}

We can see, S2 is contained within S1

We can check it very quickly by:

```
select '[1,5]']::int4range < '[3,4]']::int4range;
```

Response is: True

✔ **Overlapping element using &&:** Do two ranges share any values?

Note: it returns true if there exists any shared values between them

```
SELECT '[1, 10]']::int4range && '[10, 15]']::int4range; --t But
```

```
SELECT '[1, 10)']::int4range && '[10, 15]']::int4range; --f
```

✔ **Containment/Set-membership using @>:** Does range contain **element**?

```
SELECT '[1, 10)']::int4range @> 5; -- true
```

✔ **Containment/Set-membership using @>:** Does range contain **another range/series**?

Note: x @> Y implies if Y is contained in X

whereas, x <@ Y implies if X is contained in Y

```
SELECT '[1, 10)']::int4range @> 5; -- true
```

```
SELECT '[1, 20]']::int4range @> '[5, 12]']::int4range; -- true
```

```
SELECT '[1, 20]']::int4range <@ '[5, 12]']::int4range; -- false
```

✔ **Adjacency and Set Operations using -|-**

Note: It returns True if there is no overlapping element otherwise it returns False.

```
SELECT '[1, 5]']::int4range -|- '[5, 10]']::int4range; -- false since 5 is overlapping
```

```
SELECT '[1, 5]']::int4range -|- '(5, 10]']::int4range; --true
```

✔ **Bound Extraction Functions:**

```
1 SELECT lower('[4, 50]']::int4range); -- 4
2 SELECT upper('[1, 10]']::int4range); -- 10
3 SELECT upper('[1, 10]']::int4range); -- 11
```

Here lower () extracts the lower bound of the interval, while upper () extracts the upper bound of the interval.

4.3.6.1 A working example

Suppose you want to build an application for automation of meeting room booking.

A: Creating the Schema

```
1  -- Create the meeting rooms table
2  CREATE TABLE meeting_rooms (
3    id SERIAL PRIMARY KEY,
4    name VARCHAR(100) NOT NULL,
5    capacity INTEGER NOT NULL,
6    location VARCHAR(200)
7  );
8
9  -- Create the reservations table with a range column
10 CREATE TABLE reservations (
11   id SERIAL PRIMARY KEY,
12   room_id INTEGER REFERENCES meeting_rooms(id),
13   time_slot tsrange NOT NULL,
14   reserved_by VARCHAR(100) NOT NULL,
15   purpose TEXT,
16   created_at TIMESTAMPTZ DEFAULT NOW()
17 );
```

B: Data Insertion

```

1  -- Insert sample rooms
2  INSERT INTO meeting_rooms (name, capacity, location) VALUES
3  ('Auditorium', 300, 'Floor 1'),
4  ('Board Room', 20, 'Floor 2'),
5  ('DLT Room Library', 50, 'Floor 2');
6
7  ---- Reserve Conference Room A (ID 1) for a morning meeting
8  INSERT INTO reservations (room_id, time_slot, reserved_by, purpose)
9  VALUES (
10  1,
11  '[2026-01-30 09:00:00, 2026-01-30 10:30:00]',
12  'cse@iut.com',
13  'DBMS2 Project'
14  );
15
16  ---- Reserve the same room (ID 1) for an afternoon meeting , no conflict
17
18  INSERT INTO reservations (room_id, time_slot, reserved_by, purpose)
19  VALUES (
20  1,
21  '[2026-01-30 14:00:00, 2026-01-30 15:00:00]',
22  'swe@iut.com',
23  'Arabic Training'
24  );

```

C: Queries

```

1  --1: list of rooms for meeting
2  select id,name
3  from meeting_rooms;
4  --2: Find all bookings so far
5  SELECT id,room_id,
6  to_char(lower(time_slot), 'YY MonthYYYY HH24 MI SS') AS StartsAt, time_slot,
7  purpose
8  FROM reservations;
9
10  ---3: Find all reservations that overlap with a specific time (user given)
11
12  SELECT r.id,r.purpose, m.name as where,r.time_slot as when
13  FROM reservations r
14  JOIN meeting_rooms m ON r.room_id = m.id
15  WHERE r.time_slot && '[2026-01-30 09:15:00, 2026-01-30 11:00:00]':::tsrange;
16
17  --but it will show no row as date is changed
18  SELECT r.id,r.purpose, m.name as where,r.time_slot as when
19  FROM reservations r
20  JOIN meeting_rooms m ON r.room_id = m.id
21  WHERE r.time_slot && '[2026-03-30 09:15:00, 2026-03-30 11:00:00]':::tsrange;
22
23  -- or : it will produce no row: there is no conflict

```

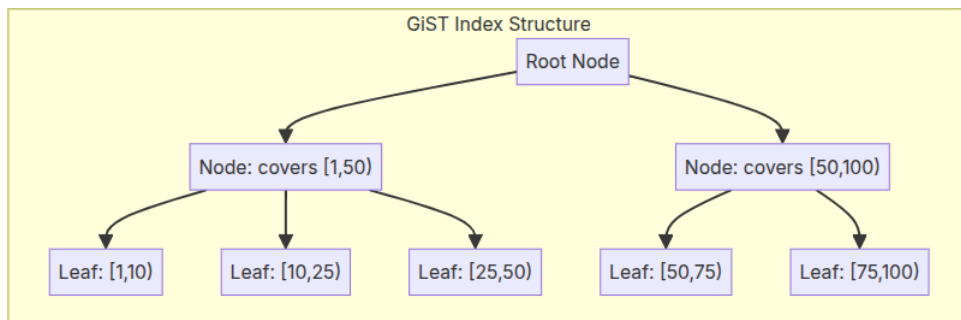


Figure 4.1: GiST Index: General Overview

```

23 SELECT r.id,r.purpose, m.name as where,r.time_slot as when
24 FROM reservations r
25 JOIN meeting_rooms m ON r.room_id = m.id
26 WHERE r.time_slot && '[2026-01-30 20:30:00, 2026-01-30 22:00:00]':::tsrange;
27

```

```

1
2 -- -- Find available rooms during a specific time slot
3 --- this will return all rows: all 3 rooms are there
4 SELECT m.*
5 FROM meeting_rooms m
6 WHERE NOT EXISTS (
7   SELECT 1 FROM reservations r
8   WHERE r.room_id = m.id
9   AND r.time_slot && '[2026-01-30 11:00:00, 2026-01-30 12:00:00]':::tsrange
10  );
11
12
13 ---- Find all reservations for today (i.e. 2026-1-31)
14 SELECT r.id,r.purpose, m.name as room_name
15 FROM reservations r
16 JOIN meeting_rooms m ON r.room_id = m.id
17 WHERE r.time_slot && '[2026-01-30, 2026-01-31]':::tsrange;
18
19 -- Get the duration of each reservation
20 SELECT
21 id,
22 reserved_by,
23 upper(time_slot) - lower(time_slot) as duration
24 FROM reservations;

```

4.3.7 GiST Indexes for Range Types

A GiST (Generalized Search Tree) index is a balanced, tree-structured framework in PostgreSQL designed for complex data types—like geometry, ranges, and full-text search—where traditional B-trees are unsuitable.

✔ Creating GiST Indexes:

```
1  --step 1:
2  -- Create a GiST index on the time_slot column
3  CREATE INDEX idx_reservations_time_slot
4  ON reservations USING GIST (time_slot);
5
6  --step 1.5: it is needed to create composite index
7  CREATE EXTENSION IF NOT EXISTS btree_gist;
8  --step 2: create a composite index
9  -- Composite GiST index for room-specific queries
10 CREATE INDEX idx_reservations_room_time
11 ON reservations USING GIST (room_id, time_slot);
12
13 --step 3: verify it works
14 EXPLAIN ANALYZE
15 SELECT * FROM reservations
16 WHERE time_slot && '[2026-01-30 09:00:00, 2026-01-30 18:00:00)>::tsrange;
```

4.3.7.1 Exclusion Constraints

At table level we can impose such constraints that we will not insert any overlapping data. Same room can not be booked by 2 customers at the same date-time.

```
1  -- Add exclusion constraint to prevent double bookings
2
3  ALTER TABLE reservations
4  ADD CONSTRAINT no_overlapping_reservations
5  EXCLUDE USING GIST (
6  room_id WITH =,
7  time_slot WITH &&
8  );
```

✔ How it works?

The code:

```
1  EXCLUDE USING GIST (
2  room_id WITH =,
3  time_slot WITH &&
4  )
```

basically tells that:

Reject the row if there is already a row where room_id is Equal (=) AND the time_slot Overlaps (&&).

📖 Now lets try to insert data:

```
1  --- not ok: as room 1 time conflicts
```

```
2  INSERT INTO reservations (room_id, time_slot, reserved_by, purpose)
3  VALUES (
4  1,
5  '[2026-01-30 10:00:00, 2026-01-30 11:30:00]',
6  'cse@iut.com',
7  'DBMS2 Project'
8  );
9  ---ok: as this room 3
10 INSERT INTO reservations (room_id, time_slot, reserved_by, purpose)
11 VALUES (
12 3,
13 '[2026-01-30 10:00:00, 2026-01-30 11:30:00]',
14 'cse@iut.com',
15 'DBMS2 Project'
16 );
```

After detecting time overlapping the data will not be inserted and it gives following error message:

```
ERROR:  conflicting key value violates exclusion constraint "no_overlapping_r
DETAIL:  Key (room_id, time_slot)=(1, ["2026-01-30 10:00:00","2026-01-
30 11:30:00"])
conflicts with existing key (room_id, time_slot)=(1, ["2026-01-30 09:00:00","
01-30 10:30:00"]).
```

Credit¹ for the topic.

¹Special Credit for this topic: [Range Types in Postgre](#)

4.3.8 Composite Data Type

A composite type represents the structure of a row or record; it is essentially just a list of field names and their data types. PostgreSQL allows composite types to be used in many of the same ways that simple types can be used.

- i.) For example, a **column of a table** can be declared to be of a composite type.
- ii.) Similarly it can be used **inside any PL/pgSQL block** just like any other variable.

Few Points:

- ✓ ROW constructor is used for data insertion
- ✓ Use VALUE to access the assigned value of the variable for imposing constraint on it
- ✓ To access individual element use (compAttributeName).elementName for Table Records
- ✓ Omit the bracket () for PL/pgSQL block

Example A: As a Table Attribute

```

1 CREATE TYPE person_name AS (
2   first_name TEXT,
3   middle_name TEXT,
4   family_name TEXT
5 );
6
7
8 Example 1: As a table attribute
9
10 CREATE TABLE customers (
11   user_id SERIAL PRIMARY KEY,
12   full_name person_name,
13   email TEXT
14 );
15
16 -- Option 1: Using the ROW constructor (Recommended for clarity): recommended
17 INSERT INTO customers (full_name, email)
18 VALUES (ROW('Alice', 'Marie', 'Smith'), 'alice@example.com');
19
20 -- Option 2: Using a string literal-not recommended
21 INSERT INTO customers (full_name, email)
22 VALUES ('("Bob", "Robert", "Jones")', 'bob@example.com');
23
24 --Select the whole object:
25 SELECT full_name FROM customers;
26
27 --Select specific sub-fields:
28 SELECT (full_name).first_name, (full_name).family_name FROM customers;
```

 The ROW() constructor is **type-agnostic**; it doesn't care what the underlying data types are.

Example:

```

1  -- 1. Create a type with numeric components
2  CREATE TYPE coordinates_3d AS (
3  x NUMERIC,
4  y NUMERIC,
5  z NUMERIC
6  );
7
8  -- 2. Create a table using that type
9  CREATE TABLE sensors (
10 sensor_id INT,
11 location coordinates_3d
12 );
13
14 -- 3. Use the ROW constructor with numbers
15 INSERT INTO sensors (sensor_id, location)
16 VALUES (101, ROW(12.5, 45.0, -1.25));
17
18 select (location).x,(location).y from sensors;
```

Example B: As local variable in PL/pgSQL Block:

```

1  DO $$
2  DECLARE
3  -- Declare the variable using the composite type name
4  v_user_name person_name;
5  v_full_text TEXT;
6  BEGIN
7  -- 1. Assign values using the ROW constructor
8  v_user_name := ROW('Leonardo', 'da', 'Vinci');
9
10 -- 2. Access individual fields using dot notation
11 -- Inside PL/pgSQL, you don't need the extra parentheses like you do in SQL
12 v_full_text := v_user_name.first_name || ' ' || v_user_name.family_name;
13
14 RAISE NOTICE 'Full Name: %', v_full_text;
15 RAISE NOTICE 'Middle Name: %', v_user_name.middle_name;
16
17 -- 3. You can also update just one part of the composite variable
18 v_user_name.first_name := 'Leo';
19
20 RAISE NOTICE 'Updated First Name: %', v_user_name.first_name;
21 END $$;
```

 **Note:** There is **no bracket** to access it, i.e. `v_user_name.first_name`

4.3.9 Domain Type

Domain types are used when a particular set of constraints need to be enforced on a base type. They are essentially a user-defined type that is based on an existing type.

 **Why Used?** In business application it is very common to have similar type of constraint for a number of entities. For instance, program values must be either of {B.Sc. , M.Sc.}

Example:

```

1  CREATE DOMAIN dept text CONSTRAINT dept_check
2  CHECK (VALUE IN ('CSE','EEE','MPE'));
3
4  CREATE TABLE DEPTS
5  (name dept,
6   location text,
7   budget numeric
8  );
9
10 insert into depts values('CSE','AB2', 2.1);
11
12 insert into depts values('CS','AB2', 2.1); -- NOT OK, violates constraints
13
14 --- Now I can create another table---
15 create table dept_achievements
16 (deptname dept,
17 description text,
18 event_when date
19 );
20
21 --- now insert data, same constraint will be reused--
22 insert into dept_achievements values('MPE', 'The event A',now());
23 insert into dept_achievements values('MPE', 'The event B',NOW() - INTERVAL '10
    months' );
24 --wrong data
25 insert into dept_achievements values('ME', 'The event A',now());

```

✔ Multiple Conditions can be Enforced in Domain

Example:

```

1  CREATE DOMAIN valid_product_code AS TEXT
2  CHECK (
3  (VALUE LIKE 'PROD-%' AND LENGTH(VALUE) = 10) -- Condition Group A
4  OR
5  (VALUE = 'INTERNAL-USE')           -- Condition Group B
6  );
7
8  CREATE TABLE inventory (
9  item_id SERIAL PRIMARY KEY,
10 item_name TEXT NOT NULL,
11 product_code valid_product_code NOT NULL -- Using our custom domain
12 );

```

```
13
14
15 -- This works (Matches 'PROD-%' AND length is 10)
16 INSERT INTO inventory (item_name, product_code)
17 VALUES ('Laptop', 'PROD-12345');
18
19 -- This works (Matches the OR condition 'INTERNAL-USE')
20 INSERT INTO inventory (item_name, product_code)
21 VALUES ('Test Bench', 'INTERNAL-USE');
22
23 -- This FAILS (Starts with PROD- but length is not 10)
24 INSERT INTO inventory (item_name, product_code)
25 VALUES ('Mouse', 'PROD-99');
26
27 -- This FAILS (Does not match either side of the OR)
28 INSERT INTO inventory (item_name, product_code)
29 VALUES ('Keyboard', 'GUEST-CODE');
```

Chapter 5

Control Structure

5.1 IF/ELSE Statement

The IF statement is used to execute a block of code if a certain condition is true.

```
1 DO
2 $$
3 BEGIN
4 IF 1 = 1 THEN
5 RAISE NOTICE 'OK';
6 END IF;
7 END;
8 $$;
9
10 ----OUTPUT-----
11 NOTICE: OK
12 DO
```

The condition can be any expression as well.

Example:

```
1 CREATE TABLE IFTEST(ID NUMERIC);
2
3 DO
4 $$
5 BEGIN
6 IF (select count(*) from IFTEST) < 1 THEN
7 RAISE NOTICE 'NO RECORDS ARE THERE IN THIS TABLE';
8 ELSE RAISE NOTICE 'THERE ARE FEW RECORDS IN THIS TABLE';
9 END IF;
10 END;
11 $$;
```

- ✔ PL/pgSQL conditional statements support EXISTS, IS NULL, and NOT operators.

Example:

```
1  --CREATE TABLE IFTEST(ID NUMERIC);
2
3  DO
4  $$
5  BEGIN
6  IF EXISTS(select * from IFTEST)
7  THEN
8  RAISE NOTICE 'THIS TABLE NOT EMPTY';
9  ELSE RAISE NOTICE 'THE TABLE IS EMPTY';
10 END IF;
11 END;
12 $$;
```

Example:

```
1  --CREATE TABLE IFTEST(ID NUMERIC);
2
3  DO
4  $$
5  DECLARE
6  XCOUNT INTEGER:=0;
7  BEGIN
8
9  SELECT COUNT(*) INTO XCOUNT
10 FROM IFTEST;
11
12 IF XCOUNT=0
13 THEN
14 RAISE NOTICE 'THIS TABLE IS EMPTY';
15
16 ELSIF XCOUNT BETWEEN 1 AND 3
17 THEN RAISE NOTICE 'THE TABLE HAS ONLY FEW RECORDS';
18
19 ELSE RAISE NOTICE 'THE TABLE HAS MORE RECORDS';
20 END IF;
21 END;
22 $$;
```

5.2 CASE Statement

One of the user-friendly features offered by PL/pgSQL is the ability to implement complex conditional logic in a concise and readable way. The CASE statement in PL/pgSQL allows users to specify different actions based on different conditions.

Example:

```
1  DO
2  $$
3
```

```

4 BEGIN
5 CASE
6 WHEN EXISTS(SELECT * FROM iftest) THEN
7 RAISE NOTICE 'table is not empty';
8 WHEN NOT EXISTS(SELECT * FROM test) THEN
9 RAISE NOTICE 'table is empty';
10 END CASE;
11 END;
12 $$;

```

More Example:

✔ Simple CASE (CASE variable WHEN)

```

1 DO
2 $$
3 DECLARE
4 XCOUNT INT:=0;
5 BEGIN
6 SELECT COUNT(*) INTO XCOUNT FROM iftest;
7 CASE XCOUNT
8 WHEN 0 THEN
9 RAISE NOTICE 'table is empty';
10 WHEN 1 THEN
11 RAISE NOTICE 'table has only one records';
12 ELSE
13 RAISE NOTICE 'table has a lot of records';
14 END CASE;
15 END;
16 $$;

```

✔ **Searched CASE** (CASE WHEN XCOUNT ...): Allows any Boolean expression, including BETWEEN, >, <, or IS NULL.

Example:

```

1 DO
2 $$
3 DECLARE
4 XCOUNT INT:=0;
5 BEGIN
6 SELECT COUNT(*) INTO XCOUNT FROM iftest;
7 CASE
8 WHEN XCOUNT = 0 THEN
9 RAISE NOTICE 'table is empty';
10 WHEN XCOUNT BETWEEN 1 AND 3 THEN
11 RAISE NOTICE 'table has only few records';
12 ELSE
13 RAISE NOTICE 'table has a lot of records';
14 END CASE;
15 END;
16 $$;

```

5.3 LOOP Statement

Example:

```
1 DO
2 $$
3 DECLARE
4 v_count INT:=0;
5 v_iteration_count INT:=1;
6 BEGIN
7 <<LoopingMark>>
8 LOOP
9 v_count:=v_count+1;
10 EXIT WHEN v_count=15;
11 v_iteration_count:=
12 v_iteration_count+1;
13 RAISE NOTICE 'The value of counter is %',v_iteration_count;
14 END LOOP LoopingMark;
15 RAISE NOTICE 'After exiting the loop the value is %',
16 v_iteration_count;
17 END;
18 $$;
```

✔ Note: In PL/pgSQL, <<LoopingMark>> is a label. Labels are identifiers enclosed in double angle brackets that allow you to name a specific block of code or a loop.

5.4 FOR Statement

PL/pgSQL provides several variants of the FOR loop statement, each designed for a specific use case.

✔ Simple FOR loop: By **default**, the FOR i IN a..b loop in PL/pgSQL is **inclusive**, meaning it always includes the upper bound (b)

```

1  DO $$
2  BEGIN
3  -- This loops from 1 to 5 inclusive
4  FOR i IN 1..5 LOOP
5  RAISE NOTICE 'Current value: %', i;
6  END LOOP;
7  END $$;
8
9  -- same output
10
11 DO $$
12 BEGIN
13 -- This loops from 1 to 5
14 FOR i IN 1..5 LOOP
15 RAISE NOTICE 'Current value: %', i;
16 END LOOP;
17 END $$;

```

📝 Quick Note in Data Insertion: You can write like:

```

1  INSERT INTO test
2  VALUES(generate_series(1, 3));
3
4  --which is equivalent to
5  INSERT INTO test (column_name)
6  VALUES (1),
7           (2),
8           (3);

```

generate_serties(a,b) is mainly used for **Generating Mock Data**.

✔ FOR opens an implicit cursor for the specified SQL query.

Example:

```

1  DO
2  $$
3  DECLARE
4  v_rec RECORD;
5  BEGIN
6  FOR v_rec IN (SELECT * FROM iftest) LOOP
7  RAISE NOTICE 'Record ... %', v_rec;
8  END LOOP;
9  END;
10 $$;

```

5.4.1 Use Case for FOR loop

Loan repayment, inserting records in iteration with basic data and others as empty.

5.5 Control Statements in PL/pgSQL: Best Practices

- **Keep Control Statements Simple**

One of the main principles of good programming is to keep things simple. This applies to control statements as well. When using control statements, you should strive to keep them as simple as possible. This means that you should avoid nesting too many control statements within each other, and you should avoid using too many conditions in a single statement. By keeping your control statements simple, you will make your code more readable and easier to understand.

- **Use Comments to Explain Complex Control Statements**

Sometimes, you may need to use complex control statements to achieve your desired result. When this happens, it is a good idea to use comments to explain what the control statement is doing. This will help other programmers who read your code to understand what is going on in the control statement.

- **Test Your Control Statements Thoroughly**

Control statements can be tricky to get right, especially if you are using complex conditions or loops. For this reason, it is essential to test your control statements thoroughly. Make sure that you test your control statements with a variety of inputs, including inputs that are outside of the expected range. This will help you to catch any bugs or errors in your code before they cause problems in production.

- **Use Meaningful Variable Names**

When writing control statements, it is essential to use meaningful variable names. This will help you to understand what the variable is used for when you come back to your code later.

Chapter 6

Functions and Procedures

6.1 Benefits of Functions and Procedure

In PostgreSQL, Functions and Procedures are both used to store reusable logic on the database server. The major benefits of using user-defined function and procedure in PostgreSQL database are as follows:

1. **Performance:**

- ✓ **Reduced network round-trips:** Instead of sending 10 queries over the wire, one function call does all the work server-side.
- ✓ **Execution plan caching:** PL/pgSQL caches query plans after the first execution, cutting repeated parsing/planning overhead.
- ✓ **Less data transfer:** Only the final result travels to the client, not intermediate rows.

2. **Encapsulation and Modularity:** Large systems often have multiple clients (web app, mobile app, internal dashboard, BI tools).

- ✓ If you write your "Calculate Discount" logic in the Python backend, the Node.js internal tool won't have access to it. The module written in java applet will re-write the same logic once again.
- ✓ By placing that logic in a Function, you ensure that every interface interacting with the data follows the exact same rules. It creates a "Single Source of Truth."

3. **Security and Access Control:**

- ✓ Grant EXECUTE on a function without exposing underlying tables at all users never have access to the raw data/code directly.
- ✓ SECURITY DEFINER functions let you run sensitive operations with elevated privileges on behalf of restricted users, in a controlled way.
- ✓ **Reduces SQL injection** surface since parameterized logic is pre-compiled.

4. **Maintainability** in Large Systems is easy.

6.2 Functions

 Create a function:

Case 1: With return statement , no out pms

```
1 CREATE OR REPLACE FUNCTION do_multiply(a numeric,b numeric)
2 RETURNS numeric
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6 RETURN a*b;
7 END;
8 $$;
```

Case 2: How to extract database values inside a local variable

Use `select a,b,c into x,y,z from test;`

```
1 CREATE OR REPLACE FUNCTION get_dbvalue(p_roomid numeric)
2 RETURNS text
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE
6 v_name text;
7 BEGIN
8 select name into v_name
9 from meeting_rooms
10 where id=p_roomid;
11 RETURN v_name;
12 END;
13 $$;
```

How to call the function:

```
1 -- stand alone
2 select get_dbvalue(3);
3
4 --- pass table fields as pms: it will show all rooms name
5 select id, get_dbvalue(id)
6 from meeting_rooms;
```

✔ Call inside another block using local variable:

Use `x=get_dbvalue(2);`

```

1 DO $$
2 DECLARE
3 x text; -- To hold the return value
4 BEGIN
5 -- This is the standard Postgres way to "receive" 3 returns
6 x=get_dbvalue(2);
7 RAISE NOTICE 'x: %', x;
8 END $$;
```

📖 PL/pgSQL parameter modes:

The parameter modes determine the behaviors of parameters. PL/pgSQL supports three parameter modes: `in`, `out`, and `inout`. A parameter takes the **in mode by default** if you do not explicitly specify it.

✔ OUT parameters:

- The out parameters are defined as a part of the argument list and are **returned as a part of the result**.
- The out parameters are very useful in functions that need to return **multiple values**.

✔ **OUT parameters** removes two lines from the code: `returns sometype` and `return` statement inside the function (at the end).

✔ Calling a function **needs only IN parameters**.

```

1 CREATE OR REPLACE FUNCTION myf1(a in numeric,OUT b numeric)
2 LANGUAGE plpgsql
3 AS $$
4 BEGIN
5 b:=a*a;
6 END;
7 $$;
8 --same function--
9 CREATE OR REPLACE FUNCTION myf1m(a in numeric,out b numeric)
10 returns numeric
11 LANGUAGE plpgsql
12 AS $$
13 BEGIN
14 b:=a*a;
15 return;
16 END;
17 $$;
18
19 --now call by:
20 select myf1(6);
```

Multiple OUT parameters:

```

1  ---v2: 2 OUT pms
2  CREATE OR REPLACE FUNCTION myf2(a IN numeric,OUT b numeric,OUT c
   numeric)
3  LANGUAGE plpgsql
4  AS $$
5  BEGIN
6  b:=a*a;
7  c:=a+a;
8  END;
9  $$;
10
11 -- call it: all
12 select myf2(9);
13 -- call it by each OUT pms
14 select b from myf2(8);
15 select c from myf2(6);
16 select b
17 ,c from myf2(34);

```

✔ **IN parameter is modifiable unlike Oracle.** which may create problem if not used properly.

```

1  CREATE OR REPLACE FUNCTION get_my_values(
2  IN input_val INT,
3  OUT main_result INT,
4  OUT p INT,
5  OUT q INT
6  ) AS $$
7  BEGIN
8  --input_val:=101;
9  main_result := input_val * 2;
10 p := input_val + 5;
11 q := input_val + 10;
12 END;
13 $$ LANGUAGE plpgsql;
14
15 -- calling it from another block
16 DO $$
17 DECLARE
18 a INT; -- To hold the "main" return
19 b INT; -- Local variable p
20 c INT; -- Local variable q
21 BEGIN
22 -- This is the standard Postgres way to "receive" 3 returns
23 SELECT main_result, p, q
24 INTO a, b, c
25 FROM get_my_values(10);
26
27 RAISE NOTICE 'a: %, b: %, c: %', a, b, c;
28 END $$;

```

6.2.1 %TYPE, %ROWTYPE and RECORD Data-types

- **%TYPE:** It is called The Column Cloner/Anchor. It provides the exact data type of a specific table column or an already declared variable.

Benefits: It is dynamic, as the base table column is changed, it gets the change effect automatically.

Example:

```
1 CREATE OR REPLACE FUNCTION get_employee_salary(emp_id INT)
2 RETURNS NUMERIC AS $$
3 DECLARE
4     -- Automatically inherits whatever type 'salary' is
5     v_salary employees.salary%TYPE;
6 BEGIN
7     SELECT salary INTO v_salary FROM employees WHERE id = emp_id;
8     RETURN v_salary;
9 END;
10 $$ LANGUAGE plpgsql;
```

- **%ROWTYPE:** It is called row cloner/anchor. It groups all the columns of a specific table into a single structured variable. You can access individual fields using dot notation (e.g., variable_name.column_name).

Benefit: It keeps your code extremely clean when dealing with full rows because you don't have to declare 15 different variables for a 15-column table. At the same time all of the fields are dynamically synced with the referenced table.

Example:

```
1 CREATE OR REPLACE FUNCTION process_employee_bonus(emp_id INT)
2 RETURNS VOID AS $$
3 DECLARE
4     -- Variable holds an entire row structured exactly like the employees
5     -- table
6     v_emp_row employees%ROWTYPE;
7 BEGIN
8     SELECT * INTO v_emp_row FROM employees WHERE id = emp_id;
9
10    -- Accessing individual columns easily
11    IF v_emp_row.performance_rating = 'Excellent' THEN
12        UPDATE employees SET salary = salary + 5000 WHERE id = v_emp_row.id;
13    END IF;
14
15    -- Functions returning VOID do not strictly require a RETURN statement,
16    -- but adding it is good practice.
17    RETURN;
18 END;
19 $$ LANGUAGE plpgsql;
```

- **RECORD:** A RECORD variable is a generic placeholder. It does not have a predefined structure when you declare it. Instead, it adapts its structure to whatever row or query result you assign to it at runtime.

Benefits: High flexibility. It can handle custom query outputs that don't match any single existing table.

Example:

```
1 CREATE OR REPLACE FUNCTION process_employee_bonus(emp_id INT)
2 RETURNS NUMERIC AS $$
3 DECLARE
4 v_emp_row employees%ROWTYPE;
5 v_final_salary employees.salary%TYPE;
6 BEGIN
7 SELECT * INTO v_emp_row FROM employees WHERE id = emp_id;
8
9 IF v_emp_row.performance_rating = 'Excellent' THEN
10 -- Update the salary and capture the new value immediately
11 UPDATE employees
12 SET salary = salary + 5000
13 WHERE id = v_emp_row.id
14 RETURNING salary INTO v_final_salary;
15 ELSE
16 -- If no bonus, the final salary is just the current salary
17 v_final_salary := v_emp_row.salary;
18 END IF;
19
20 RETURN v_final_salary;
21 END;
22 $$ LANGUAGE plpgsql;
```

Another example:

```
1  --a simple demo of record data type
2
3  CREATE TABLE DEPTS
4  (ID INTEGER PRIMARY KEY,
5   NAME TEXT,
6   LOCATION TEXT,
7   BUDGET NUMERIC
8  );
9
10 CREATE TABLE EMPS
11 (EID SERIAL PRIMARY KEY,
12  NAME TEXT,
13  DOB DATE,
14  DEPT INTEGER REFERENCES DEPTS(ID)
15 );
16
17 -- INSERT DATA
18 INSERT INTO DEPTS(ID,NAME,LOCATION,BUDGET)
19     VALUES(4,'CSE','AB2',3.1),(2,'EEE','AB1',2.5),(3,'MPE','AB1',1.2);
20
21 INSERT INTO EMPS(NAME,DOB,DEPT)
22     VALUES('ABDUL KARIM',CURRENT_DATE - INTERVAL '30 years',4),('ABDUL
    RAZZAK',CURRENT_DATE - INTERVAL '51 years',4),
    ('AKHTAR HOSSAIN',CURRENT_DATE - INTERVAL '41 years',2),('ABDUL
    HALIM',CURRENT_DATE - INTERVAL '23 years',3);
```

```
1  -- now create a function
2
3  CREATE OR REPLACE FUNCTION record_demo(p_eid INT)
4  RETURNS VOID AS $$
5  DECLARE
6  emp_rec RECORD;
7  BEGIN
8
9  SELECT
10 e.eid EID,
11 e.name ENAME,
12 e.dob EDOB,
13 d.name DEPT,
14 d.location DLOCATION into emp_rec
15 FROM EMPS AS e
16 INNER JOIN DEPTS AS d
17 ON e.DEPT = d.ID
18 where e.eid=p_eid;
19
20 RAISE WARNING 'Given ID is: %', p_eid;
21
22 RAISE WARNING 'Given ID with Name: %', emp_rec.ename;
23
24 RAISE WARNING 'his dob is : %', emp_rec.edob;
25
26 RAISE WARNING 'His Dept Loaction is : %', emp_rec.dlocation;
27
28 END;
29 $$ LANGUAGE plpgsql;
```

6.2.2 Return More Rows

PL/pgSQL function can easily return multiple row.

There are two standard ways to achieve this, depending on how you want to build the result set:

1. **Option (a): RETURN QUERY** The most efficient and common way. It executes a SQL query and immediately appends all of its results to the function's output.
2. **Option (b): RETURN NEXT** Used when you need to loop through data and programmatically build the result set row-by-row.

Implementation: By **Option (a)**:

It is done by using RETURNS TABLE with RETURN QUERY

This is the cleanest approach when you are simply wrapping a SELECT query inside a function, potentially with some business logic or dynamic parameters.

By defining RETURNS TABLE (...), you specify the exact columns the function will output.

Example:

```
1  -- Create a mock table for the example
2  CREATE TABLE IF NOT EXISTS employees (
3  emp_id INT,
4  emp_name VARCHAR(50),
5  department VARCHAR(50),
6  salary NUMERIC
7  );
8
9  -- The Function
10 CREATE OR REPLACE FUNCTION get_employees_by_dept(p_department VARCHAR)
11 RETURNS TABLE (id INT, name VARCHAR, current_salary NUMERIC) AS $$
12 BEGIN
13 -- Evaluate business logic here if needed...
14
15 -- Execute the query and return all matching rows at once
16 RETURN QUERY
17 SELECT emp_id, emp_name, salary
18 FROM employees
19 WHERE department = p_department;
20
21 -- You can also have multiple RETURN QUERY statements in one function!
22 END;
23 $$ LANGUAGE plpgsql;
```

✔ How to call it?

Because it returns multiple rows, you treat the function exactly like a table or a view.

```
SELECT * FROM get_employees_by_dept('CSE');
```

Chapter 7

Exception Handling in PL/pgSQL

When writing database functions, things will go wrong. A user might try to register with an email that already exists, a calculation might accidentally divide by zero, or a foreign key constraint might fail.

Without exception handling, any error will completely abort your transaction, roll back all previous changes, and throw an ugly error message at your application. Exception handling allows you to intercept these errors, log them, fix them on the fly, or return a clean, user-friendly message.

 Types of Exceptions: We have classified exception in 2 main groups:

1. **Built-in Exceptions:** Exception due to errors such as database constraints violation, divide by zero and so on. It is generally done using `EXCEPTION ... WHEN ...` clause
2. **User-defined Exceptions:** Apart from these basic sets of errors in Business Applications it is possible to flag/signal an error due to the violation of user-defined business logic. It is mainly handled by `RAISE EXCEPTION` clause.

Both types will be explored using examples.

7.1 Built-in Exceptions

Scenario

We want a function that adds a new user to a members table. If the email already exists, we don't want the application to crash; we want the function to gracefully return -1.

Lets create the required table first:

```
1 CREATE TABLE members (  
2   id SERIAL PRIMARY KEY,  
3   email VARCHAR(255) UNIQUE,  
4   joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
5 );
```

Note: email must be unique as per the DDL. Now if you try to insert duplicate email address by:

```
insert into members(email) values('b@iut.com');
```

The database will through errors something like below:

```
ERROR:  duplicate key value violates unique constraint "members_email_key"
DETAIL:  Key (email)=(b@iut.com) already exists.
```

But by exception handling capability it can be better managed.

Now instead of the direct insertion we will **use a function** with exception handling mechanism.

```
1 CREATE OR REPLACE FUNCTION register_member(p_email VARCHAR)
2 RETURNS INT AS $$
3 DECLARE
4 v_new_id INT;
5 BEGIN
6 INSERT INTO members (email)
7 VALUES (p_email)
8 RETURNING id INTO v_new_id;
9
10 RETURN v_new_id;
11
12 EXCEPTION
13 -- Intercept duplicate key errors on the unique email column
14 WHEN unique_violation THEN
15 RAISE WARNING 'Attempted to register duplicate email: %', p_email;
16 RAISE WARNING 'Please try with another email other than : %', p_email;
17
18 RETURN -1;
19 END;
20 $$ LANGUAGE plpgsql;
```

Now insert data through function:

```
select register_member('a@iut.com');
```

✔ **More Error Information:** GET DIAGNOSTICS command: We can use **GET STACKED DIAGNOSTICS** command to extract deeply specific debugging details, like the column name that caused the issue. When an error is caught, PostgreSQL provides special automatic variables like SQLSTATE (the error code) and SQLERRM (the error message).

Modified Function:

```
1 CREATE OR REPLACE FUNCTION register_member2(p_email VARCHAR)
2 RETURNS INT AS $$
```

```

3  DECLARE
4  v_new_id INT;
5  v_err_context TEXT;
6  BEGIN
7  INSERT INTO members (email)
8  VALUES (p_email)
9  RETURNING id INTO v_new_id;
10
11  RETURN v_new_id;
12
13  EXCEPTION
14  WHEN OTHERS THEN
15  -- FIX: Added the keyword "STACKED" right here
16  GET STACKED DIAGNOSTICS v_err_context = PG_EXCEPTION_CONTEXT;
17
18  -- Logging the error details
19  RAISE WARNING 'Error Code: %, Message: %', SQLSTATE, SQLERRM;
20  RAISE WARNING 'Context: %', v_err_context;
21
22  RETURN -1;
23
24  END;
25  $$ LANGUAGE plpgsql;

```

Now error are dealt with the following message if you issue
`select register_member2('a@iut.com');`

```

WARNING: Error Code: 23505, Message: duplicate key value violates
         unique constraint "members_email_key"
WARNING: Context: SQL statement "INSERT INTO members (email)
VALUES (p_email)
RETURNING id"
PL/pgSQL function register_member2(character varying)
line 6 at SQL statement

```

Note: It exactly points the error in your code (with line number), it is specially helpful when error is not very obvious.

Error Message vs. Code:

In the exception section we have so far used `WHEN unique_violation THEN` where `unique_violation` is a pre-defined condition. It has a similar error code such as "23505" for `unique_violation`

Same code can be written as:

```
WHEN SQLSTATE '23505' THEN
```

The following Table ¹ summarizes major error codes:

¹See Documentation for more.

Condition Name	Error Code	Timing
unique_violation	23505	A UNIQUE or PRIMARY KEY constraint is breached.
foreign_key_violation	23503	When FK violates
not_null_violation	23502	Attempts to insert a NULL into a restricted column
division_by_zero	22012	When some value is divided by 0
no_data_found	P0002	A SELECT INTO query returns zero rows
too_many_rows	P0003	A SELECT INTO query returns multiple rows but the variable only holds one

7.2 User-defined Exceptions

Scenario: Banking

Here one customer may wish to withdraw an amount of money, but his current balance must be greater than his requested amount.

Lets create the table (only 1 will work here for this discrete example)

```

1  create table accounts(
2  acc_no serial primary key,
3  name text,
4  balance numeric(10,2)
5  );
6
7  %Lets insert 2 records
8  insert into accounts(name,balance) values('Mr. Abdul Karim',4000),('Mr.
   Bob',9000);

```

```

1  CREATE OR REPLACE FUNCTION process_withdrawal(p_account_id INT, p_amount
   NUMERIC)
2  RETURNS VOID AS $$
3  DECLARE
4  v_current_balance NUMERIC;
5  BEGIN
6  -- 1. Fetch the current balance (assuming an 'accounts' table exists)
7  SELECT balance INTO v_current_balance
8  FROM accounts
9  WHERE acc_no = p_account_id;
10
11 -- 2. Check business logic and RAISE custom exception if it fails
12 IF v_current_balance < p_amount THEN
13 RAISE EXCEPTION 'Transaction failed: You DO NOT have sufficient funds. Your
   Balance is %', v_current_balance

```

```
14 USING ERRCODE = 'U0101'; -- 'U' signifies a User-defined code
15 END IF;
16
17 -- 3. Proceed with the update if no exception was raised
18 UPDATE accounts
19 SET balance = balance - p_amount
20 WHERE acc_no = p_account_id;
21
22 RAISE NOTICE 'Withdrawal Successfully Done!';
23
24 EXCEPTION
25 -- 4. Catch our specific user-defined exception using its SQLSTATE
26 WHEN SQLSTATE 'U0101' THEN
27 RAISE WARNING 'Handled User Defined Error: % %', SQLSTATE, SQLERRM;
28 -- You can log the error to an audit table here, or take other actions.
29
30 -- 5. Always good practice to catch unexpected errors
31 WHEN OTHERS THEN
32 RAISE WARNING 'An unexpected error occurred: % (SQLSTATE: %)', SQLERRM,
33 SQLSTATE;
34 END;
35 $$ LANGUAGE plpgsql;
```

Now call it by:

```
select process_withdrawal(1,100); -- success
select process_withdrawal(1,8100); -- error
```

WARNING: Handled User Defined Error (U0101): Transaction failed: You DO NOT
Your Balance is 3700.00

Note: Here SQLSTATE is the error code assigned and SQLERRM is the associated error message (human readable)

Chapter 8

Cursor to Handle Selected Records

A cursor in PostgreSQL is a database object that allows you to retrieve and process a result set one row at a time, rather than loading the entire dataset into memory at once. Think of it as a pointer or an iterator that moves through a table's rows sequentially.

Cursors are highly useful in PL/pgSQL (PostgreSQL's procedural language) when handling massive datasets that would otherwise cause memory exhaustion if loaded all at once.

There are 2 types of cursors:

1. Implicit Cursor
2. Explicit Cursor

8.1 Implicit Cursor

An implicit cursor is automatically created and managed by PostgreSQL whenever you execute a query that returns multiple rows inside a PL/pgSQL block, such as a FOR loop. No need to open, fetch, or close it manually.

Properties:

- **Automatic Lifecycle:** PostgreSQL manages the allocation and deallocation of memory.
- **Ease of Use:** They require much less steps to use.
- **Scope:** The cursor only exists for the duration of the loop.

Example: Suppose we have a students table containing names and marks. We want to loop through all students and print their status.

Actually: in real-world there is a need to inspect all selected record one by one and do something based on the finding (example given here is only for demonstration purpose)

Example: Implicit Cursor

```

1  CREATE OR REPLACE FUNCTION print_employees(p_id integer)
2  RETURNS void AS $$
3  DECLARE
4  -- Variable to hold each row implicitly fetched
5  v_emp RECORD;
6  BEGIN
7  -- The FOR loop implicitly creates, opens, fetches, and closes the cursor
8  FOR v_emp IN
9  SELECT eid,name ,dob
10 FROM emps
11 WHERE eid <= p_id
12 LOOP
13 RAISE NOTICE 'Emp ID: % Emp Name: %, Emp DOB: %',v_emp.eid,
14             v_emp.name,v_emp.dob;
15 END LOOP;
16 END $$ LANGUAGE plpgsql;
17
18 -- now call it by:
19 select print_employees(3);

```

I will print information for employees with id<=3

 **Where to use?** Implicit cursors are best tool for "Simple row-by-row looping loops" where it "Moves strictly forward, row-by-row"

 **GET DIAGNOSTICS ... ROW_COUNT**

Obtains the number of rows processed by the most recent SQL statement. **Example:**

```

1  UPDATE employee
2  SET salary = salary * 1.15
3  WHERE dept_id = 10;
4
5  GET DIAGNOSTICS v_count = ROW_COUNT;
6
7  RAISE NOTICE 'Rows updated: %', v_count;

```

8.2 Explicit Cursor

An explicit cursor must be manually declared, opened, fetched, and closed by the programmer. It gives you total control over how rows are processed and allows you to pass parameters dynamically.

Steps involved:

1. **DECLARE**: Define the cursor name and its associated query.
2. **OPEN**: Execute the query and allocate memory for the result set.
3. **FETCH**: Retrieve the current row into variables and advance the pointer.
4. **CLOSE**: Release the allocated memory resources.

Example: Students CGPA based short-listing

1. Data Insertion Part

```
1  --- demo explicit cursor
2  create table students
3  (id serial primary key,
4   name text,
5   cgpa numeric(4,2)
6  );
7
8  insert into students (name,cgpa)
   values('a',3.2),('b',3.62),('c',3.85),('d',3.92),('e',3.42),('f',3.83);
```

2. Function using Ex Cursor

```

1  --- now create a function with ex cursor--
2  CREATE OR REPLACE FUNCTION find_top_students(p_min_cgpa numeric)
3  RETURNS void AS $$
4  DECLARE
5  -- 1. DECLARE the explicit cursor (passing through the function parameter)
6  cur_top_stu CURSOR(min_cgpa numeric) FOR
7  SELECT name, cgpa FROM students WHERE cgpa >= min_cgpa order by cgpa desc;
8
9  -- Variables to store the fetched data
10 v_name students.name%TYPE;
11 v_cgpa students.cgpa%TYPE;
12 BEGIN
13 -- 2. OPEN the cursor and pass the input argument value
14 OPEN cur_top_stu(p_min_cgpa);
15
16 LOOP
17 -- 3. FETCH the current row data into our variables
18 FETCH cur_top_stu INTO v_name, v_cgpa;
19
20 -- Exit the loop if no more rows are found
21 EXIT WHEN NOT FOUND;
22
23 RAISE NOTICE 'Top Student: % got cgpa of % ', v_name, v_cgpa;
24 END LOOP;
25
26 -- 4. CLOSE the cursor to free memory
27 CLOSE cur_top_stu;
28
29 END $$ LANGUAGE plpgsql;

```

Example: Ex Cursor with Scroll Option:

Key Navigation Commands for Scrollable Cursors:

- FETCH LAST: Instantly jumps to the end of the query.
- FETCH PRIOR (or FETCH BACKWARD): Moves the pointer up one row.
- FETCH FIRST: Instantly rewinds the pointer to the very beginning.
- FETCH ABSOLUTE n: Jumps to a specific row number (e.g., FETCH ABSOLUTE 5 jumps to the 5th row in the dataset).
- MOVE: Moves the pointer but does not return data. This is much faster if you just want to skip rows without loading their values into memory.

Example: 1. Data Insertion

```

1  -- table and data
2  create table citizen
3  (cid serial primary key,
4   name text,
5   income numeric
6  );
7  insert into citizen(name,income)
8  values('a',30000),('b',10000),('c',43000),('d',70000),('e',96000),('f',32000)
9  ,('g',78000),('h',80000);

```

2. Function with Ex Cursor Scroll option:

```

1  CREATE OR REPLACE FUNCTION navigate_citizen_data()
2  RETURNS void AS $$
3  DECLARE
4  -- 1. Explicitly add the "SCROLL" keyword to the declaration
5  cur_citi SCROLL CURSOR FOR
6  SELECT name,income from citizen order by income desc;
7
8  v_name citizen.name%TYPE;
9  v_income citizen.income%TYPE;
10 BEGIN
11 -- 2. Open the cursor (the pointer starts before the first row)
12 OPEN cur_citi;
13
14 -- 3. Jump to the very last row in the query result
15 FETCH LAST FROM cur_citi INTO v_name, v_income;
16 RAISE NOTICE 'Last in the List:Name is %, Income is : %', v_name, v_income;
17
18 -- 4. Move backward exactly one row from our current position
19 FETCH PRIOR FROM cur_citi INTO v_name, v_income;
20 RAISE NOTICE 'One but Last of the List:Name is %, Income is : %', v_name,
21   v_income;
22
23 -- 5. Jump all the way back to the very first row
24 FETCH FIRST FROM cur_citi INTO v_name, v_income;
25 RAISE NOTICE 'Top of the List:Name is %, Income is : %', v_name, v_income;
26
27 -- 6. Skip forward 3 rows WITHOUT fetching the data into variables
28 MOVE FORWARD 3 IN cur_citi;
29
30 -- 7. Fetch the current row after our jump
31 FETCH NEXT FROM cur_citi INTO v_name, v_income;
32 RAISE NOTICE 'After skipping 3 rows:Name: %, Income: %', v_name, v_income;
33
34 -- 8. Always close your explicit cursors!
35 CLOSE cur_citi;
36 END $$ LANGUAGE plpgsql;
37
38 -- now call it by:

```

```
38 select navigate_citizen_data();
```

8.2.1 Cursor FOR Loop

In PostgreSQL, a FOR loop can automatically manage a cursor to evaluate a query and evaluate its result rows sequentially. Using a FOR loop **eliminates** the need to write manual OPEN, FETCH, and CLOSE commands because PL/pgSQL handles the cursor lifecycle **automatically**.

Example:

```
1 CREATE OR REPLACE FUNCTION check_low_stock_alerts(p_threshold INT)
2 RETURNS void AS $$
3 DECLARE
4 -- 1. Declare the explicit cursor
5 cur_low_stock CURSOR FOR
6 SELECT product_name, stock_quantity
7 FROM products
8 WHERE stock_quantity < p_threshold;
9
10 -- 2. Declare a record variable to hold each row's data
11 v_product RECORD;
12 BEGIN
13 RAISE NOTICE '--- LOW STOCK ALERT START ---';
14
15 -- 3. The FOR loop automatically OPENS the cursor,
16 -- FETCHES each row, and CLOSSES it at the end.
17 FOR v_product IN cur_low_stock LOOP
18
19 RAISE NOTICE 'Item: % | Only % left in stock!',
20 v_product.product_name,
21 v_product.stock_quantity;
22
23 END LOOP;
24
25 RAISE NOTICE '--- LOW STOCK ALERT END ---';
26 END $$ LANGUAGE plpgsql;
```

A better example:

Here we will convert our previous function `find_top_students()`; using Cursor FOR Loop:

```
1 CREATE OR REPLACE FUNCTION find_top_studentsv2(p_min_cgpa numeric)
2 RETURNS void AS $$
3 DECLARE
4 -- 1. DECLARE the explicit cursor (remains the same)
5 cur_top_stu CURSOR(min_cgpa numeric) FOR
6 SELECT name, cgpa FROM students WHERE cgpa >= min_cgpa ORDER BY cgpa DESC;
7
8 -- 2. Instead of individual variables, declare a single RECORD loop variable
9 v_student RECORD;
10 BEGIN
11 -- 3. The FOR loop automatically OPENS the cursor, passes the parameter,
12 -- FETCHES each row, and safely CLOSES it when finished.
13 FOR v_student IN cur_top_stu(p_min_cgpa) LOOP
14
15 -- Access data cleanly using dot notation (record_variable.column_name)
16 RAISE NOTICE 'Top Student: % got cgpa of % ', v_student.name, v_student.cgpa;
17
18 END LOOP;
19 END $$ LANGUAGE plpgsql;
20
21 --- now cal it--
22 select find_top_studentsv2(3.6);
```